

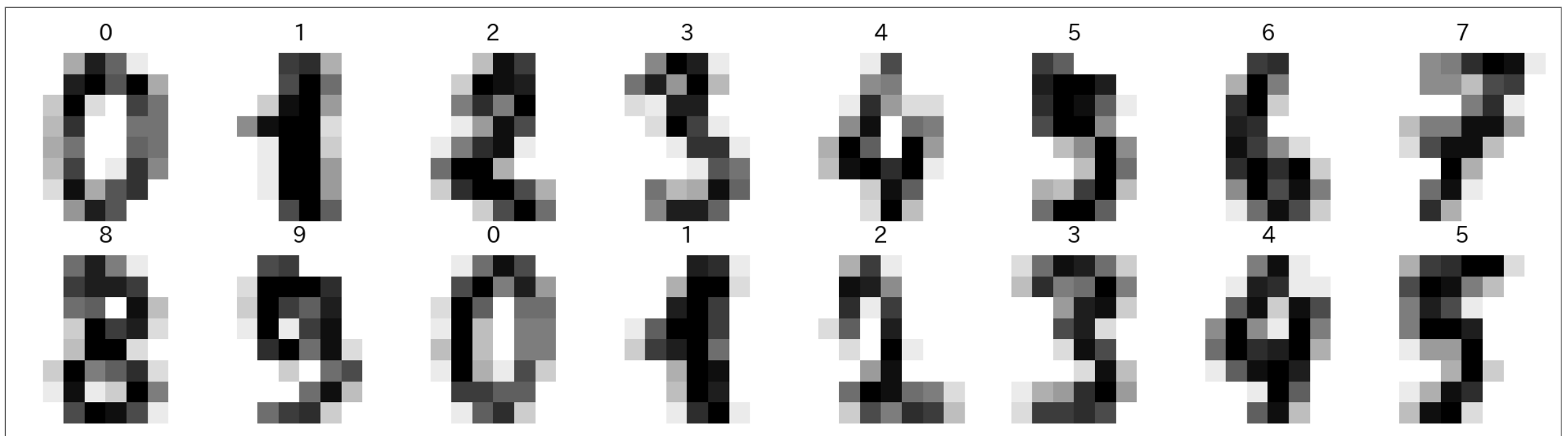
手書き数字分類プロジェクト

約束の回収

rkskmt

1

初回の約束、覚えているか



最終回までに、この仕組みを **中身から全部** 自分の手で組み立てる。ブラックボックスは残さない。

- 今日が最終回。 **手書き数字を読み取る機械** を完成させる
- 必要な部品は、ここまでの13回で **すべて自作済み**

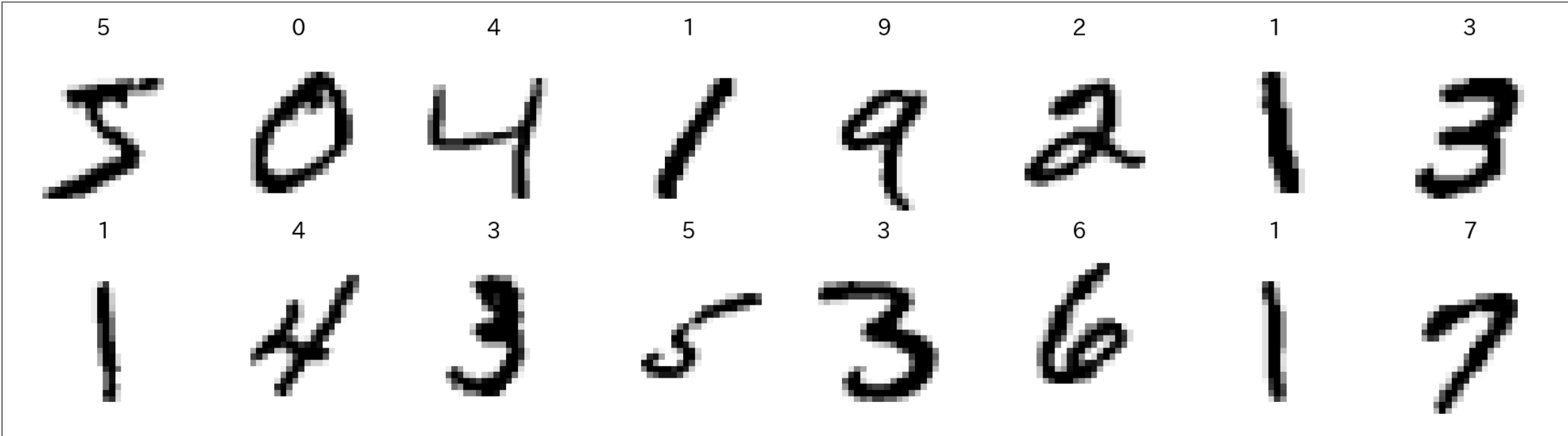
2

データ：MNIST

- 手書き数字の定番データセット。 **訓練6万枚・テスト1万枚**、各28×28ピクセル

```
1 import torch
2 import torch.nn as nn
3 from torchvision import datasets, transforms
4
5 transform = transforms.ToTensor() # 画像 → 0~1 の tensor
6 train_ds = datasets.MNIST(root="data", train=True, download=True, transform=transform)
7 test_ds = datasets.MNIST(root="data", train=False, download=True, transform=transform)
8
9 print("訓練:", len(train_ds), "枚 / テスト:", len(test_ds), "枚")
10 img, label = train_ds[0]
11 print("1枚の形:", tuple(img.shape), " 正解ラベル:", label)
```

訓練：60000 枚 / テスト：10000 枚
1枚の形：(1, 28, 28) 正解ラベル：5



3

画像は784次元のベクトル

- $28 \times 28 = 784$ **個の数字の並び** — モデルにとって画像は784次元のベクトル
- 魚は2次元（輝度・重さ）だった。次元の呪いの回で遊んだ高次元が、**本物の入力**としてやってきた
- **nn.Flatten()** が「画像 → ベクトル」の変換をやる

① ノート

784次元でも、やることは魚の2次元と **完全に同じ**： 特徴量ベクトル → 変換を重ねる → 確率 → 損失 → 勾配降下。

4

出力層を付け替える：10クラス分類

- 答えは Salmon / Sea bass の2択ではなく、**0～9 の10択**
- 「変換を重ねる」の回の表のとおりに、**出力層を付け替える**だけ

問題	出力の活性化関数
2クラス分類	シグモイド（1個の確率）
多クラス分類	ソフトマックス（10個の確率分布）

$$\text{softmax}(u_k) = \frac{e^{u_k}}{\sum_j e^{u_j}}$$

10個の出力を「合計1の確率」に変換

- 損失は多クラス版の交差エントロピー：正解クラスの確率 $p_{\text{正解}}$ に対して $-\log p_{\text{正解}}$
- PyTorch では **nn.CrossEntropyLoss** がソフトマックスごとやってくれる（打ち消しの恩恵もそのまま）

モデル：見慣れた3層

```
1 torch.manual_seed(0)
2
3 model = nn.Sequential(
4     nn.Flatten(),           # (28,28) → 784
5     nn.Linear(784, 128),    # アフィン変換
6     nn.ReLU(),             # 折る
7     nn.Linear(128, 10),     # 出力層（10クラスぶんのスコア）
8 )
9
10 n_params = sum(p.numel() for p in model.parameters())
11 print(f"パラメータ総数: {n_params:,}")
```

パラメータ総数: 101,770

- 魚の最小単位は**3個**だった。いまや**10万個** — それでも学習ループは同じ3拍子

① Adam — 最適化回の伏線回収

勾配降下法の回で名前だけ出た「学習率を自動で加減する改良版」。中身は勾配降下法そのもの（勾配の使い方が少し賢いだけ）。大きなネットでは標準装備。

ミニバッチ：6万枚は一口ずつ

- 6万枚まとめて勾配を計算すると重い → **128枚ずつ** 取り出して1歩ずつ進む
- データを一巡することを **1エポック** と呼ぶ

```
1 train_loader = torch.utils.data.DataLoader(train_ds, batch_size=128, shuffle=True)
2 test_loader  = torch.utils.data.DataLoader(test_ds,  batch_size=1000)
3
4 loss_fn     = nn.CrossEntropyLoss()
5 optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
```

- NN最小単位の回で「魚を1匹ずつ見せる」
とやったのと同じ発想（あちらはバッチサイズ1）

7

学習する

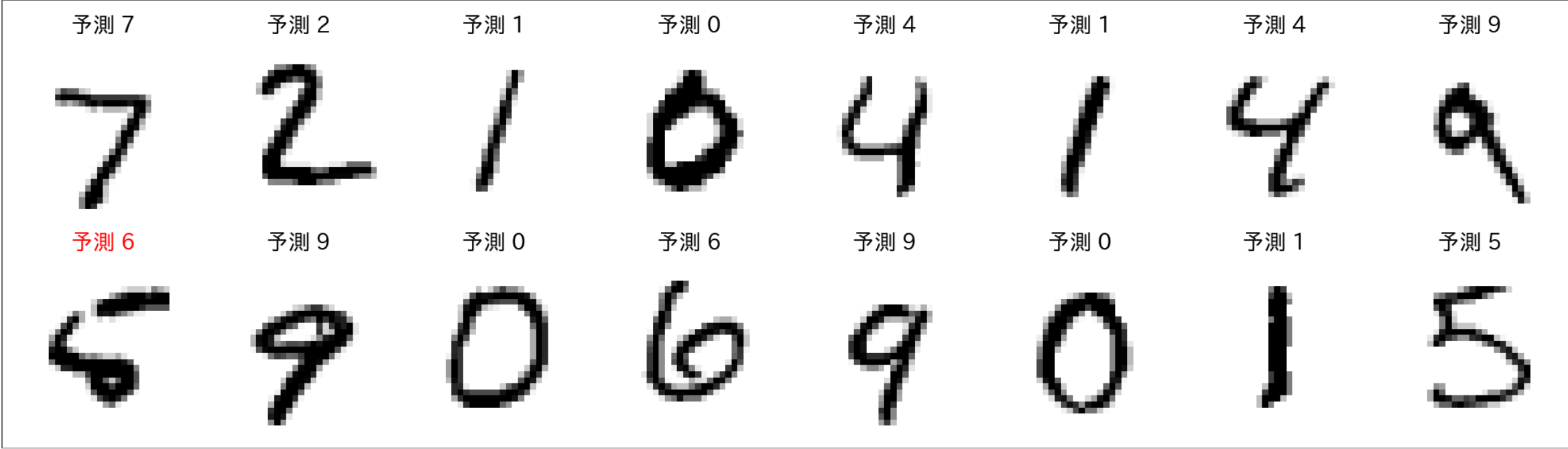
```
1 for epoch in range(1, 4):
2     for x_batch, y_batch in train_loader:
3         optimizer.zero_grad()
4         loss = loss_fn(model(x_batch), y_batch)
5         loss.backward()
6         optimizer.step()
7
8     # テストデータ（見ていない1万枚）で採点 - 過学習と汎化の回の鉄則
9     correct = 0
10    with torch.no_grad():
11        for x_batch, y_batch in test_loader:
12            correct += (model(x_batch).argmax(dim=1) == y_batch).sum().item()
13    print(f"エポック {epoch} | テスト正答率: {correct / len(test_ds):.2%}")
```

エポック 1 | テスト正答率: 93.50%
エポック 2 | テスト正答率: 95.14%
エポック 3 | テスト正答率: 96.33%

- 見たことのない手書き1万枚を、**96%以上**の精度で読めるようになった
- エポック数や中間の変換の数を増やすと、どこまで伸びるか試してみよう

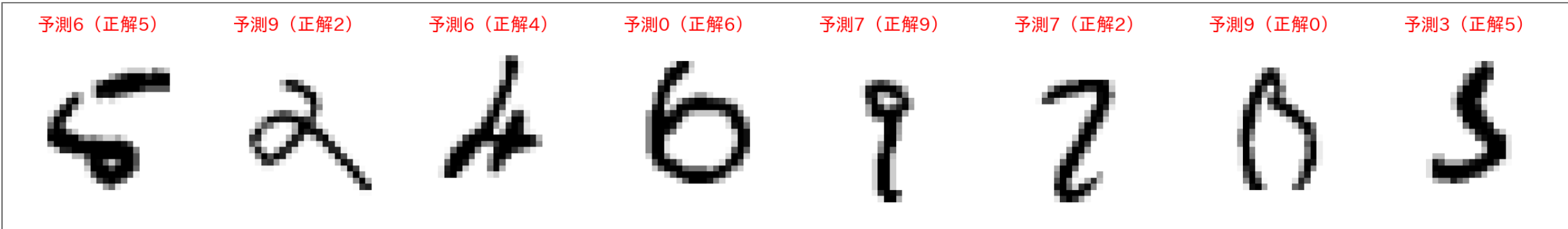
8

読めている様子



- 予測ラベルは **モデルの出力**。初回に「謎」として見せた光景が、いま手元のコードで起きている

間違いを眺める



- 間違えた字をよく見ると、**人間でも迷うものが多い**
- 精度の数字だけでなく **間違いの中身** を見るのが、モデルと付き合うコツ（混同行列の発想）

約束は果たされたか：部品の棚卸し

この機械の中に、説明できない部品が残っていないか確認する。

部品	正体	学んだ回
<code>nn.Linear</code>	アフィン変換（傾きと切片の一般化）	決定境界／NN最小単位
<code>nn.ReLU</code>	空間を折る非線形	変換を重ねる
ソフトマックス＋交差エントロピー	微分しやすい損失	NN最小単位／今回
<code>loss.backward()</code>	連鎖律の自動化	バックプロパゲーション
<code>Adam</code>	勾配降下法の改良版	最適化と勾配降下法
テストデータで採点	汎化を測る鉄則	過学習と汎化

❶ 約束の回収

ブラックボックス、ゼロ。 全部品を手で作るか、手計算で検算済み。

この先の世界

- 画像がもっと大きく複雑になったら？
 - 784次元を全部つなぐのは無駄が多い → 近くのピクセルだけ見る **畳み込み (CNN)**
- 入力文章だったら？
 - 単語同士の関係に重みをつける **Attention** → **Transformer** → いまの **大規模言語モデル (LLM)**
- どちらも今日の機械と同じ原理の上に立つ：**変換を重ねて、損失を決めて、勾配で学習する**

💡 ヒント

名前は新しくても、開け方はもう知っている。「forward は何か」「損失は何か」「勾配はどう流れるか」 — この3つを聞けばいい。

12

講義全体のまとめ

- 魚の選別から始まり、**fit()** という1行の中身を開け続けた
 - しきい値 → 直線 → 木 → ロジスティック回帰 → 多層NN
 - 損失（交差エントロピー・MSE）と勾配降下法が全編を貫く背骨
- 満点を疑うこと（過学習）、スケールを揃えること、テストで採点すること
- 最後に、**手書き数字を読む機械**を自分の部品で組み上げた

📌 持ち帰り課題（任意）

紙に数字を書いて撮影し、28×28 のグレースケールに縮小して読ませてみよう：

```
1 from PIL import Image
2 import numpy as np
3 import torch
4
5 img = Image.open("my_digit.png").convert("L").resize((28, 28))
6 x = 1.0 - np.array(img, dtype=np.float32) / 255.0 # MNISTは白地に黒ではなく黒地に白
7 x = torch.tensor(x).unsqueeze(0).unsqueeze(0) # (1, 1, 28, 28)
8 print("予測:", model(x).argmax(dim=1).item())
```

うまく読めないときは、なぜかを考えるところからが本番（ヒント：訓練データと自分の字の「分布」の違い）。

13